

Introducere în Prolog

1 Obiective

Lucrarea de față vă familiarizează cu principalele concepte din limbajul Prolog: fapte, reguli și întrebări. Pe lângă acestea se vor prezenta: tipurile de date utilizate în Prolog și regulile de unificare.

2 Considerații teoretice

2.1 Paradigma logică

Paradigma de programare este un stil fundamental de programare care dictează:

- Modul de **reprezentare** a datelor (ex: fapte, variabile, clase etc.)
- Modul de **prelucrare** a datelor (ex: asignări, comparații, evaluări etc.)

Paradigma logică face parte din paradigma declarativă. Un limbaj de programare declarativ răspunde la întrebarea **CE** trebuie să facă un program. Să considerăm următorul exemplu în limbajul declarativ SQL:

```
SELECT nume, prenume FROM Studenți WHERE an = 3
```

Instrucțiunea de mai sus nu îi spune interpretorului SQL cum să realizeze căutarea, ci îi spune doar ce condiție trebuie să respecte rezultatul. Prolog este un limbaj de programare logic, deci implicit și declarativ.

În opoziție, un limbaj de programare imperativ (ex: C, Pascal, C++, C#, Java etc.) răspunde la întrebarea **CUM** trebuie să rezolve programul o anumită problemă.

2.2 Concepte Prolog

Instrucțiunile în Prolog sunt reprezentate de fapte și reguli. După ce programul sursă Prolog (*.pl) a fost consultat de interpretor (meniu *File -> Consult*) se pot pune întrebări. Răspunsul la întrebări se determină pe baza faptelor și a regulilor folosind tehnica backtracking-ului.

Tot ceea ce nu este cunoscut sau nu poate fi demonstrat este considerat fals.

Comentariile pe o linie încep cu simbolul %, pe mai multe linii se încadrează între `/*` și `*/`

2.3 Fapte

Faptele sunt predicate care sunt tot timpul adevărate. Se mai numesc și axiome (din matematică). Pot avea zero, unu sau mai multe argumente. Aritatea unui predicat reprezintă numărul de argumente al acelui predicat.

Exemple:

```
suntem_in_sala_108.  
animal(elefant).  
animal('antilopa Gnu').  
inaltime(girafa, 5.5). % înălțimea îi în metrii  
arbore( t(1, t(-2, nil, nil), t(8,nil,nil)) ).
```

2.4 Tipuri de date

Simple:

- constante
 - o numere (ex: 47, 6.3 etc.)
 - o simboluri/atom (ex: girafa, 'Romania', 'antilopa Gnu' etc.)
 - Dacă au caractere speciale trebuie înconjurate de ghilimele simple
- variabile
 - o Încep cu litera mare sau cu _ (ex: X, Aux, _12 etc.)
 - o Un singur _ reprezintă o variabilă anonimă/liberă

Compuse:

- structuri (ex: t(1, t(-2, nil, nil), t(8,nil,nil)) etc.)
 - o liste (ex: [], [1, 2, 3], [1, 2 | _] etc.)
 - [] = lista vidă
 - Lista [1,2,3] este reprezentată intern prin '!(1, '!(2, '!(3, []))
 - Șablonul [H|T] împarte lista în H=elementul din capul listei și T=lista ce conține restul elementelor
 - o string-uri (ex: "Hello World")

2.5 Întrebări

Întrebările pot fi văzute ca și scopuri ale programului Prolog. Răspunsul la o întrebare poate fi afirmativ sau negativ. Dacă folosim variabile în întrebare putem obține și alte informații.

Exemple:

```
?- suntem_in_sala_108.  
true.
```

```
?- animal('antilopa Gnu').
true.
?- animal(X).
X = elefant ; % repetă întrebarea cu ; sau n sau spațiu
X = 'antilopa Gnu' ;
false. % nu mai există un alt animal definit în program
```

2.6 Unificare

Simbolul = realizează unificarea dintre cei 2 termeni.

Valoare este orice tip de dată care nu este/nu conține o variabilă neinițializată.

Termen 1	Termen 2	Operație
valoare	valoare	comparație
valoare	variabila neinițializată	asignare
variabila neinițializată	valoare	asignare
variabila neinițializată	variabila neinițializată	variabilele devin sinonime

2.7 Reguli

Regula reprezintă definiția unui predicat sub forma clauzei de tip Horn:

$p(\dots)$ dacă $p_{11}(\dots)$ și $p_{12}(\dots)$ și ... și $p_{1n}(\dots)$.

...

$p(\dots)$ dacă $p_{m1}(\dots)$ și $p_{m2}(\dots)$ și ... și $p_{mk}(\dots)$.

$p(\dots)$ = capul clauzei (maxim un predicat)

$p_{11}(\dots)$ și ... = corpul clauzei (zero, unu sau mai multe predicate)

fapt = clauză fără corp

întrebare = clauză fără cap

Simboluri speciale în Prolog:

`:-` = „dacă”

`,` = „și”

`;` = „sau” (poate fi realizat și prin scrierea mai multor reguli pentru același predicat)

Exemple:

```
mai_inalt(X,Y) :- inaltime(X,Hx), inaltime(Y,Hy), Hx>Hy.
```

```
% prima data luam înălțimea lui X, apoi luam înălțimea lui Y
```

```
% și în final verificăm inegalitatea
```

```
drum(X,Y) :- muchie(X,Y).
```

```
drum(X,Y) :- muchie(X,Intermediar), drum(Intermediar,Y).
```

```
% drumul între nodurile X și Y poate să fie conexiunea directă
```

```
% între cele 2 noduri SAU dacă nu există conexiune directă ne
```

```
% folosim de o conexiune indirectă printr-un nod intermediar
```

3 Exerciții

1. Ce tip de date sunt următorii termeni:

- | | | | | | |
|------|------|------|---------|----|--------------------|
| a. | X | d. | hello | g. | [a, b, c] |
| b. | 'X' | e. | Hello | h. | [A, B, C] |
| - c. | _138 | - f. | 'Hello' | i. | [Ana, are, 'mere'] |

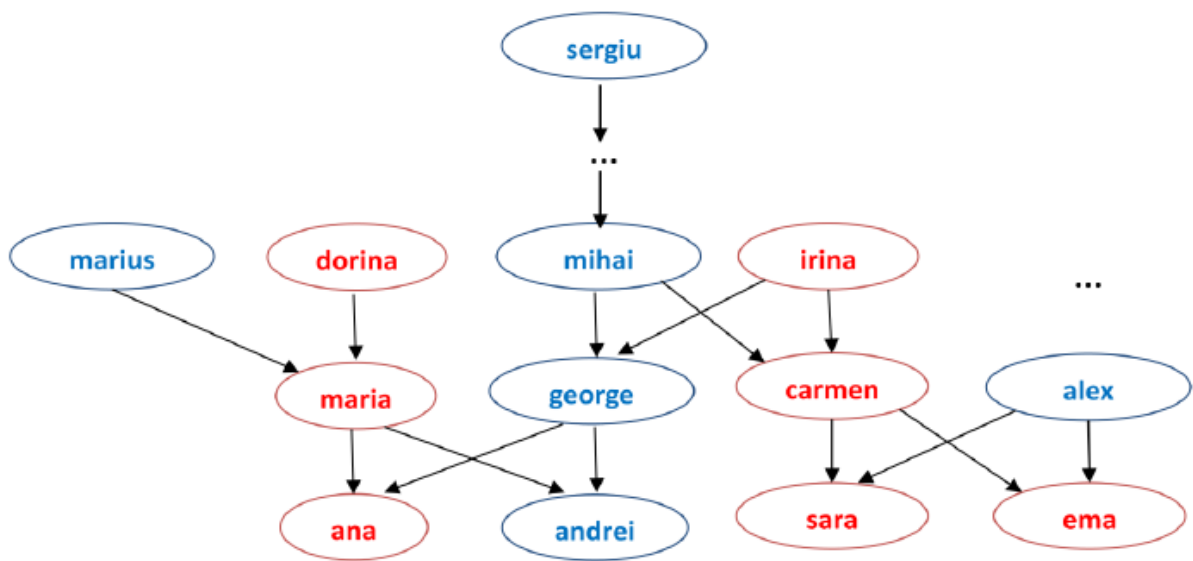
2. Ce fac următoarele predicate predefinite în Prolog:

```
var(Term),  
nonvar(Term),  
number(Term),  
atom(Term),  
atomic(Term),  
list(Term)
```

3. Testează următoarele unificări

- a. ?- a = a.
- b. ?- a = b.
- c. ?- 1 = 2.
- d. ?- 'ana' = 'Ana'.
- e. ?- X = 1, Y = X.
- f. ?- X = 3, Y = 2, X = Y.
- g. ?- X = 3, X = Y, Y = 2.
- h. ?- X = ana.
- i. ?- X = ana, Y = 'ana', X = Y.
- j. ?- a(b,c) = a(X,Y).
- k. ?- a(X,c(d,X)) = a(2,c(d,Y)).
- l. ?- a(X,Y) = a(b(c,Y),Z).
- m. ?- tree(left, root, Right) = tree(left, root, tree(a, b, tree(c, d, e))).
- n. ?- k(s(g),t(k)) = k(X,t(Y)).
- o. ?- father(X) = X.
- p. ?- loves(X,X) = loves(marsellus,mia).
- q. ?- [1, 2, 3] = [a, b, c].
- r. ?- [1, 2, 3] = [A, B, C].
- s. ?- [abc, 1, f(x) | L2] = [abc | T].
- t. ?- [abc, 1, f(x) | L2] = [abc, 1, f(x)].

4. Scrieți noi predicate pentru relațiile de rudenie.



Figură 1. Arbore genealogic

% Predicatul woman/1

```
woman(ana).
woman(sara).
woman(ema).
woman(maria). % ...
```

% Predicatul man/1

```
man(andrei).
man(george).
man(alex). % ...
```

% Predicatul parent/2

```
parent(maria, ana). % maria este părintele anei
parent(george, ana). % george este părintele anei
parent(maria, andrei).
parent(george, andrei). % ...
```

% Predicatul mother/2

```
mother(X,Y) :- woman(X), parent(X,Y).
% X este mama lui Y, daca X este femeie și X este părintele lui Y
```

Programul sursa (ex: genealogy.pl) poate fi consultat folosind meniul din interpretor: File -> consult sau se poate scrie în interpretor următorul predicat predefinit:

```
?- consult('C:/Users/student/Desktop/genealogy.pl').
true. % Dacă nu există nici o eroare va returna true
```

4.1. Testați următoarele întrebări:

```
?- man(george). % este george bărbat?  
true.  
?- man(X). % cine este bărbat?  
X = andrei ? ; % repetăm întrebarea cu ; sau n sau spațiu  
X = george ? ;  
X = alex ? ;  
false.  
  
?- parent(X, andrei). % Cine sunt părinții lui andrei?  
X = maria ? ;  
X = george ? ;  
false.  
?- parent(maria, X). % Cine sunt copii mamei?  
X = ana ? ;  
X = andrei ? ;  
false.  
  
?- mother(ana, X). % Cine sunt copii mamei?  
false.  
?- mother(X, ana). % Care este mama mamei?  
X = maria ? ; % repetăm întrebarea = mai are ana o alta mamă?  
false.
```

4.2. Scrieți predicatul father/2

4.3. Completați predicatele man/1, woman/1 și parent/2 pentru a acoperi arborele genealogic de mai sus.

4.4. Testați următoarele întrebări:

```
?- father(alex, X).  
?- father(X, Y).  
?- mother(dorina, maria).
```

4.5. Testați următoarele predicate:

% Predicatul sibling/2

% X și Y sunt frați/surori dacă au același părinte și X diferit de Y
sibling(X,Y) :- parent(Z,X), parent(Z,Y), X\=Y.

% Predicatul sister/2

% X este sora lui Y dacă X este femeie și X și Y sunt frați/surori
sister(X,Y) :- sibling(X,Y), woman(X).

% Predicatul aunt/2

% X este mătușa lui Y dacă este sora lui Z și Z este părintele lui Y
aunt(X,Y) :- sister(X,Z), parent(Z,Y).

4.6. Scrieți predicatele `brother/2`, `uncle/2`, `grandmother/2` și `grandfather/2`.

4.7. Urmăriți pașii pentru găsirea răspunsului la următoarele întrebări:

- ?- `aunt(carmen, X)`.
- ?- `grandmother(dorina, Y)`.
- ?- `grandfather(X, ana)`.

Pentru a activa urmărirea scrieți:

- ?- `trace`.

Pentru a dezactiva urmărirea scrieți:

- ?- `notrace`.

4.8. Scrieți predicatul `ancestor/2`. X este strămoșul lui Y dacă X este legat de Y printr-o serie de relații de tip părinte.